

CSCE 649 PHYSICALLY-BASED MODELING · MODULE 03 · LECTURE NOTES

Particle Systems and Springs

The simplest system, and most of cloth already.

MODULE OUTCOMES

- Implement a particle system with forces, emitters, and lifetimes.
- Derive the damped spring force and explain why damping must be relative.
- Build a mass-spring cloth with structural, shear, and bending springs.
- Explain why mass-spring models are fundamentally limited.
- Choose spring constants from material behaviour rather than by guessing.

1 Particles

A particle is a position, a velocity, and a mass, plus whatever the effect requires: colour, age, size, temperature. The state vector is the concatenation of every particle's position and velocity, and the derivative function computes forces and divides by mass.

```
void ParticleSystem::derivative(const double* x, double t, double* dx) {
    unpack(x);
    std::fill(force.begin(), force.end(), vec3(0));
    for (auto& f : force_generators) f->apply(pos, vel, force); // accumulate
    for (int i = 0; i < n; i++) {
        dx_pos[i] = vel[i];
        dx_vel[i] = force[i] * inv_mass[i];
    }
}
```

STORE INVERSE MASS, NOT MASS

This looks like a micro-optimisation and is not. **A particle with infinite mass — one that is pinned in place — has an inverse mass of exactly zero.**

Multiplying the force by zero produces zero acceleration, with no branch, no special case, and no risk of a division by infinity.

The payoff compounds. Constraint solvers (Module 04) weight corrections by inverse mass, and a pinned particle automatically receives none. Collision response (Module 06) divides impulses by the sum of inverse masses, and a collision with static geometry falls out as the case where one term is zero. Rigid bodies (Module 07) use inverse inertia tensors for exactly the same reason.

Choosing this representation at the start removes a surprising amount of special-case code later, and every physics engine does it.

Force	Expression	Notes
Gravity	$F = m \mathbf{g}$	Proportional to mass, so acceleration is independent of it — everything falls at the same rate, as it should.
Linear drag	$F = -k \mathbf{v}$	Cheap, and physically correct only at very low Reynolds number. Often used anyway as a stabiliser.
Quadratic drag	$F = -k \mathbf{v} \mathbf{v}$	The correct form for air at ordinary speeds. Noticeably different for fast-moving particles.
Spring	§2	The one that requires care.
Global viscous damping	$F = -k \mathbf{v}$ on everything	The simplest stabiliser and the easiest to overuse. It removes energy from every mode indiscriminately, and a system with enough of it looks like it is moving underwater. Students reach for it to fix instability; it works, and it is almost always the wrong fix.
Wind, vortices, turbulence	Procedural fields	Art direction rather than physics, and entirely legitimate (Module 01).

2 Springs

$$\mathbf{F} = -k (|\mathbf{d}| - L) \mathbf{d} - k_d (\mathbf{v}_{\text{rel}} \cdot \mathbf{d}) \mathbf{d}$$

where $\mathbf{d}$ is the vector between the endpoints, L the rest length, $\mathbf{d}$ the unit vector along it, and $\mathbf{v}_{\text{rel}}$ the relative velocity of the endpoints. The force is applied **equally and oppositely** to the two ends.

THREE WAYS THE DAMPING GOES WRONG, ALL COMMON

Damping absolute velocity rather than relative. The spring then resists motion through space, so a chain falling freely under gravity slows down for no reason. Linear momentum is not conserved, and the momentum plot from Module 01 shows it immediately.

Damping the full relative velocity rather than its component along the spring. This resists the endpoints moving *past* each other as well as toward and away, which means it resists rotation about the spring axis. A swinging pendulum made of such springs comes to rest for no physical reason, and cloth stops moving unnaturally quickly. **The projection onto $d\mathbf{r}$ is what prevents this**, and it is the detail most often omitted.

Applying the force to only one endpoint. Momentum is created from nothing and the whole assembly drifts across the screen.

All three produce motion that looks superficially plausible, which is why they survive. All three are caught instantly by plotting total momentum, which is what the Module 01 harness is for.

3 Mass-spring cloth

Cloth, at its simplest, is a grid of particles connected by three kinds of spring. It is worth implementing even though Module 09 will explain why it is theoretically unsound, because almost every real-time cloth system is some descendant of it.

Spring type	Connects	Resists	Without it
Structural	Each particle to its four immediate grid neighbours.	Stretching and compression.	The cloth has no integrity at all.
Shear	Each particle to its four diagonal neighbours.	In-plane shearing.	The grid collapses diagonally like a parallelogram linkage — a square of fabric flattens into a line.
Bending	Each particle to the neighbours two steps away.	Folding out of plane.	The cloth folds with no resistance and takes sharp creases that no fabric would.

Disable each type in turn and watch what fails. It takes ten minutes and explains the roles better than any description.

THE CENTRAL DIFFICULTY: CLOTH SHOULD BARELY STRETCH

Real fabric extends by a few percent under its own weight, no more. Reproducing that with springs requires a very high structural stiffness.

High stiffness forces a tiny time step — Module 02's bound $\Delta t < 2\sqrt{(m/k)}$ — and the cost of a simulation scales accordingly.

Low stiffness gives visible stretching, and the result is the classic 'super-elastic' mass-spring look: a tablecloth that behaves like a sheet of rubber, sagging under its own weight in a way no fabric does.

There is no stiffness value that is both affordable and correct. This single tension drives three later modules: implicit integration (Module 02 §3) makes high stiffness affordable, position-based constraints (Module 04) sidestep stiffness entirely, and strain limiting (Module 10) enforces the inextensibility directly. All three are responses to this problem.

3.1 Why mass-spring models are limited

- **Behaviour depends on the mesh.** Rotate the triangulation by 45° and the material's stiffness changes direction with it. Real cloth has properties independent of how you chose to discretise it.
- **There is no notion of volume.** A mass-spring cube can be turned inside out — every spring can be at its rest length in an inverted configuration — and nothing in the model objects. Deformable solids built this way collapse.
- **Spring constants are not material parameters.** You cannot look up the spring constant of silk. You tune until it looks right at one resolution, and the tuning does not transfer.

- **Anisotropy is accidental.** Fabric genuinely is anisotropic — warp and weft behave differently — but in a mass-spring model the anisotropy comes from the grid rather than from the material, so it cannot be specified.
- **Module 09 fixes all of this** with continuum mechanics, where the parameters are measurable material properties and the behaviour converges as the mesh refines. Mass-spring remains ubiquitous because it is simple, fast, and adequate when art-directed.

4 Choosing parameters

The usual approach — picking spring constants by trial until the motion looks acceptable — produces a simulation that breaks the moment anything changes. These three relations make the parameters mean something.

Relation	Why
$k \propto 1/L$ for structural springs.	A finer mesh has shorter edges. If k is held constant, a subdivided mesh is a <i>softer</i> material, which is wrong. Scaling k inversely with rest length keeps the macroscopic stiffness fixed.
$m = \rho A/n$, distributing real mass by area.	Assigning every particle a mass of 1 means a finer mesh is a heavier cloth. Distribute the actual total mass across the particles.
$\zeta = k_d / (2\sqrt{(km)})$ — parameterise by damping <i>ratio</i> .	The raw damping constant has no meaning independent of k and m . The damping ratio does: $\zeta = 1$ is critically damped and looks lifeless; ζ between 0.05 and 0.3 is the useful range for most fabric. The value transfers across resolutions and materials.

TEST AT THREE RESOLUTIONS, EVERY TIME

Run the same scenario at coarse, medium, and fine mesh resolution and compare the results.

The gross behaviour must be similar. The fine mesh will have more detail — smaller folds, finer wrinkles — but the cloth should drape to roughly the same shape and settle in roughly the same time.

If the fine mesh behaves like a different material, your parameters are resolution-dependent and the simulation is not a model of anything. It is a one-off tuned to a single mesh.

This will happen the first time you try it, and it is the most common defect in a hand-tuned mass-spring system. It is also how an artist will eventually find it — by subdividing a mesh for a close-up shot and discovering the cloth has changed character, three days before the deadline.

5 Particles as an effects tool

- **Emitters** define spawn rate, spawn region, and the distribution of initial velocities. **Most of a particle effect's character is decided here**, not in the simulation.

- **Lifetimes.** Particles age, fade, and are recycled. Use a free list and a fixed-capacity pool — allocating per particle at scale is the single most common performance mistake.
- **Unphysical forces.** Vortex rings, attractors, and **curl noise** for turbulence. Curl noise is the standard trick: take the curl of a noise field and the result is divergence-free by construction, which gives swirling, fluid-looking motion without simulating a fluid at all.
- **Rendering order.** Transparent particles must be drawn back-to-front, which means a per-frame sort by view depth. At a million particles this is a real cost and often the bottleneck.
- **Data layout.** Millions of particles is routine. Store structure-of-arrays rather than array-of-structures — the force loop touches positions and velocities and nothing else, and the difference in cache behaviour is several times.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Baraff & Witkin — Particle System Dynamics (free notes)	Section C of the course pack. Force generators, the architecture of §1, and spring forces done correctly.
Provot — Deformation Constraints in a Mass-Spring Model (1995, free)	The original three-spring cloth model of §3, and the strain limiting idea that Module 10 develops.
Ten Minute Physics — cloth and softbody episodes	Working mass-spring cloth with code, and a clear demonstration of the stretching problem.
Bridson, Fedkiw & Anderson — Robust Treatment of Collisions, Contact and Friction for Cloth (free)	Where mass-spring cloth has to go to become production-usable. Read before Module 10.
Robert Bridson — curl-noise for procedural flow (free)	The turbulence technique in §5. Short paper, immediately useful.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a particle system with gravity and both linear and quadratic drag. Compare the terminal velocity of each against the analytic value.
- 2 Implement a damped spring. Verify that total momentum is conserved for a free-floating chain of ten particles — the plot should be flat to floating-point precision.

- 3 Deliberately implement each of the three damping errors from §2 and show what each does to the momentum plot and to the motion.
- 4 Build a mass-spring cloth with all three spring types. Disable each type in turn and record what fails.
- 5 Find the maximum stable time step as a function of structural stiffness. Determine the stiffness at which the cloth stretches less than 5% under its own weight, and report the time step that requires.
- 6 Implement the parameter scaling of §4 and run the three-resolution test. Report whether the drape matches.
- 7 Deliberately break the scaling — fix k regardless of edge length — and show how the material changes with resolution.
- 8 Sweep the damping ratio from 0.01 to 1.0 and record the qualitative behaviour at each value. Identify the usable range for yourself.
- 9 Implement curl noise turbulence for a particle effect and compare against plain Perlin noise applied as a force. The difference is striking.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why store inverse mass rather than mass? Give three places it pays off.
- 2 Write the damped spring force and say what each term does.
- 3 Give three ways spring damping is commonly implemented incorrectly and the symptom of each.
- 4 Name the three cloth spring types and what fails without each.
- 5 Why is cloth's inextensibility a problem for mass-spring models?
- 6 Give four fundamental limitations of mass-spring models.
- 7 Why must k scale with edge length, and what goes wrong otherwise?
- 8 What is the damping ratio, and what range is useful?
- 9 What is the three-resolution test and what does failing it mean?